Development Team Design Document
System Technical Analysis & Reporting
~ STAR ~

MSCS710 Project
Spring 2020

This design document is prepared for use by the STAR development team. Its purpose is to provide guidance for the design, build, and test of the product, ultimately enabling a successful deployment for the customer.

Version	Author(s)	Description	Date
1.0	William Martin & Kaitlin Roffe	Initial Version	Feb 26, 2020

Contents

Introduction **3**

Overview **4**

Approach 4

Development Methodology 5

Computing Resources 5

Development **6**

Metrics Gathering 6

Database Configuration 6

Database Schema 7

User Interface 8

STAR System Dashboard Concept 9

Package Build & Distribution 10

Installation 10

Accessibility 10

Supporting Material 10

Risks & Dependencies **11**

References **11**

Introduction

The System Technical Analysis & Reporting [STAR] product supports users of the Linux operating system by providing information about the machine being used, and the operating system and processes that it is running. STAR displays the state of the system, its components, and the programs and processes running at any given time.

STAR is compatible with the Ubuntu Linux distribution, and will display system information in the following categories:

- CPU
  - Processor type
  - Processor speed

- Memory
  - Total installed
  - Currently available

- Disk
  - Total available
  - Currently available

- Program / process information
  - Running processes
  - Process list history

The STAR user interface will exist as a web application, presenting content in a web browser window to the user as a dashboard view. This will allow for the display of system information on a variety of client devices, including tablets and mobile phones. STAR will operate using a client/service model, with components installed on the server being monitored, and with client [user-side] access provided in a standard web browser. The system dashboard may be displayed on any system using a standard TCP/IP connection, either on the local network or accessing the server remotely. Data collection will take place even if connectivity is lost between server and client. Upon reconnection, the user will have access to all data collected during any network outage. Security features will be present that only allow authorized users to access the system dashboard.

Overview

The following key topics are laid out in this document:

1. Approach
    - High-level breakdown of the internal and external design.
2. Development
    - Design considerations for the application components, and the tools that will be used to assist in development.
3. Distribution
    - The method of distribution for the application, and to whom it will be accessible.
4. Risks & Dependencies
    - Outlines internal and external dependencies, the risks they pose, and how risks will be mitigated.

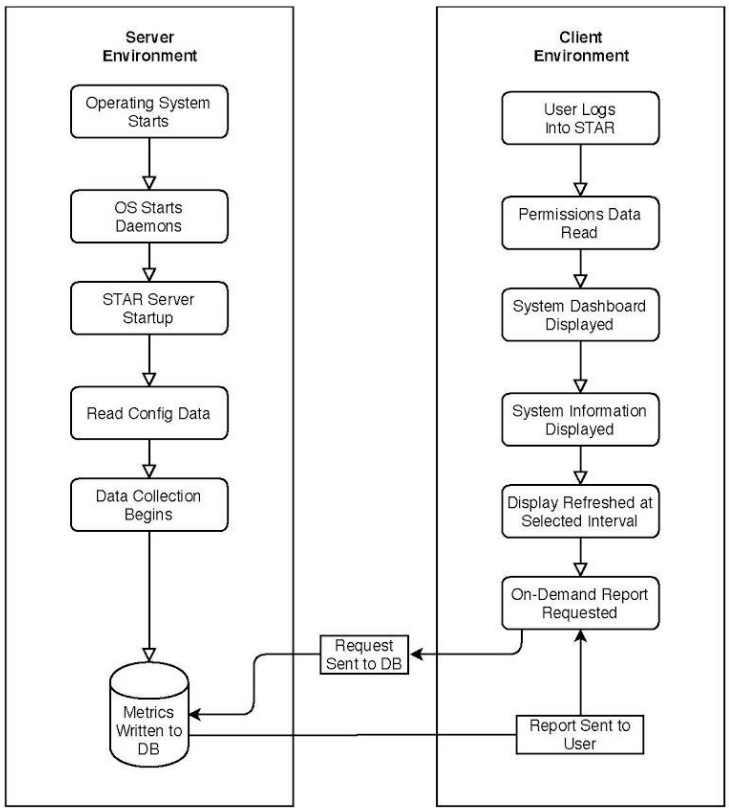
Approach

STAR works by collecting various system information [metrics], storing them in a database, and providing ways to display this information:

1. System Dashboard - provides real-time display of system information.

2. System Reporting - a reports module that provides access to data previously collected and stored.

STAR supports metrics collection on the Linux OS, with initial support for Ubuntu Linux. Other distributions may be supported in the future. A web interface provides access to system metrics from any platform. This provides a flexible deployment method, as it does not restrict customers to a specific platform. Choice of client device may include desktop, laptop/portable PC, tablet, or mobile phone.



Development Methodology

Development for STAR will follow the Agile approach. Overall product objectives are defined in this design document. The product evolves through numerous iterations during the build design, development, and process. Lessons learned and problems resolved feed back into the process, for a system of continuous improvement. The overall goal of taking this approach is to deliver better quality software, and to do so faster, with incremental builds acting as proof of concept along the way.

Computing Resources

Computing Resources used in the development and use of STAR are as follows:

Customer environment:

Server: Any hardware capable of running a supported Linux distribution.

Client: No specific operating system is required.
Choice of client hardware may include:

- Personal Computer
- Tablet
- Mobile / Smartphone

Development team environment

Server: LAMP Stack - Linux / Apache / mySQL / PHP
R Programming Language

Front-End: HTML5 & CSS3
Bootstrap - used to assist in the creation of System Dashboard.
Popper Bootstrap add-on - to provide data elements for UI visual display.

Dev. PCs: OS: Windows, Mac, and Linux are supported.
IntelliJ IDEA Integrated Development Environment.
RStudio - aid for development of R code and crontab files.
Dreamweaver - possible IDE backup to IntelliJ, if needed.

Dev / Build: This process is managed using the IntelliJ IDE.
Source code repository & version control is provided by GitHub remote repository.
GitHub interaction is provided using IntelliJ, or via command line, if needed.
Defect tracking is provided by the GitHub Issues defect tracking system.

Development

Metrics Gathering

The purpose of this application is to provide a way for users to monitor the performance of their system. The metrics that have been identified as key to this solution are: CPU, Memory, and Disk utilization. Additionally, the user will be provided with the top twenty processes using the most memory on the system, to assist with investigations and system performance tuning.

Various performance indicators will be read using the following native Linux commands:

- lscpu
- cat /proc/meminfo
- df -h --total
- top

The use of native Linux functionality to obtain desired system information means STAR is less likely to have future compatibility concerns than if it relied on a third party package.

The R language will be used to read metrics from the Linux OS and write them to a mySQL database. R is a free, open source language used for data manipulation and calculations. The system() command will be used to run Linux commands, and data frames will be used to manipulate data into tables that can be pushed to the database using an ODBC connection.

Cron will be used to schedule and run the R script at predetermined intervals. The cron daemon is a job scheduler in Linux, with jobs specified in crontab text files in /etc/crontab. The job will automatically be kicked off at system boot.

Cron has its own built-in logging, which will inform the customer that a collection operation was run. Additional logging will be created in the R script to show the outcome of attempts to read the system metrics and write them to the database.

Database Configuration

A mySQL database has been selected to store the system metrics. MySQL was chosen since it is free and open source software, and part of the ubiquitous LAMP (Linux, Apache, mySQL, PHP/Perl/Python) stack; meaning its components will be widely known and available, and the development environment mirrored by many customer environments. The database will contain four tables, one for each of the system commands being used. The tables will each use a timestamp as their unique identifier. At this time, we do not envision the need for views in the

database, since data will be aggregated in real time for the web app. The data in these tables will be periodically pruned, so as not to exceed a history further back than web client views permit, or customer needs dictate.

Database Schema

The STAR server database uses the following mySQL tables. The primary key for each table is the same [dateTime], and is shown once for all tables.

CPU Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**dateTime**	datetime			No	*None*		
2	**cpuMHz**	decimal(8,3)			No	*None*		
3	**cpuMHzMin**	decimal(8,3)			No	*None*		
4	**cpuMHzMax**	decimal(8,3)			No	*None*		

Memory Info Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**dateTime**	datetime			No	*None*		
2	**memTotal**	int(10)			No	*None*		
3	**memFree**	int(10)			No	*None*		

Disk Info Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**dateTime**	datetime			No	*None*		
2	**size**	decimal(5,1)			No	*None*		
3	**used**	decimal(5,1)			No	*None*		
4	**avail**	decimal(5,1)			No	*None*		
5	**usePercent**	tinyint(3)			No	*None*		

Process Info Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**dateTime**	date			No	*None*		
2	**pid**	mediumint(8)			No	*None*		
3	**user**	varchar(20)	utf8_unicode_ci		No	*None*		
4	**cpuPercent**	decimal(4,0)			No	*None*		
5	**memPercent**	decimal(4,0)			No	*None*		
6	**command**	varchar(40)	utf8_unicode_ci		No	*None*		

Table primary keys are same for all tables:

Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
PRIMARY	BTREE	Yes	No	dateTime	0	A	No	

The database will additionally contain a table of user logins. This table will be used to authenticate access to the web application, and to program administrative tasks. This table will be the only database component that accepts user created, vs system generated data. To limit direct database access, and prevent insertion of malicious instructions into code being executed [SQL injection], all user input will be sanitized prior to use, and parameterized prepared queries will be used for all user-initiated database access, as this binds data to a prepared SQL statement at query runtime.

Users Table

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra
1	**user**	varchar(20)	utf8_unicode_ci		No	*None*		
2	**password**	varchar(256)	utf8_unicode_ci		No	*None*		
3	**admin**	char(1)	utf8_unicode_ci		No	*None*		
4	**email**	varchar(64)	utf8_unicode_ci		No	*None*		
5	**phone**	varchar(20)	utf8_unicode_ci		No	*None*		
6	**description**	varchar(50)	utf8_unicode_ci		No	*None*		

User Interface

As noted earlier, the STAR user interface may be displayed on any standard personal computer, tablet, or mobile phone. The user interface is provided using standard web browser software. As such, no special programs need to be installed on the client device, other than what is required to display any standard web page.

When a customer calls up the STAR program web client, the following occurs:

1. A request for updated data is sent from the client UI to the server running the STAR program.
2. If not already done, the user must authenticate by logging into the program.
3. Data is then obtained from the system using various commands, read from the STAR database, and used to populate the System Dashboard.

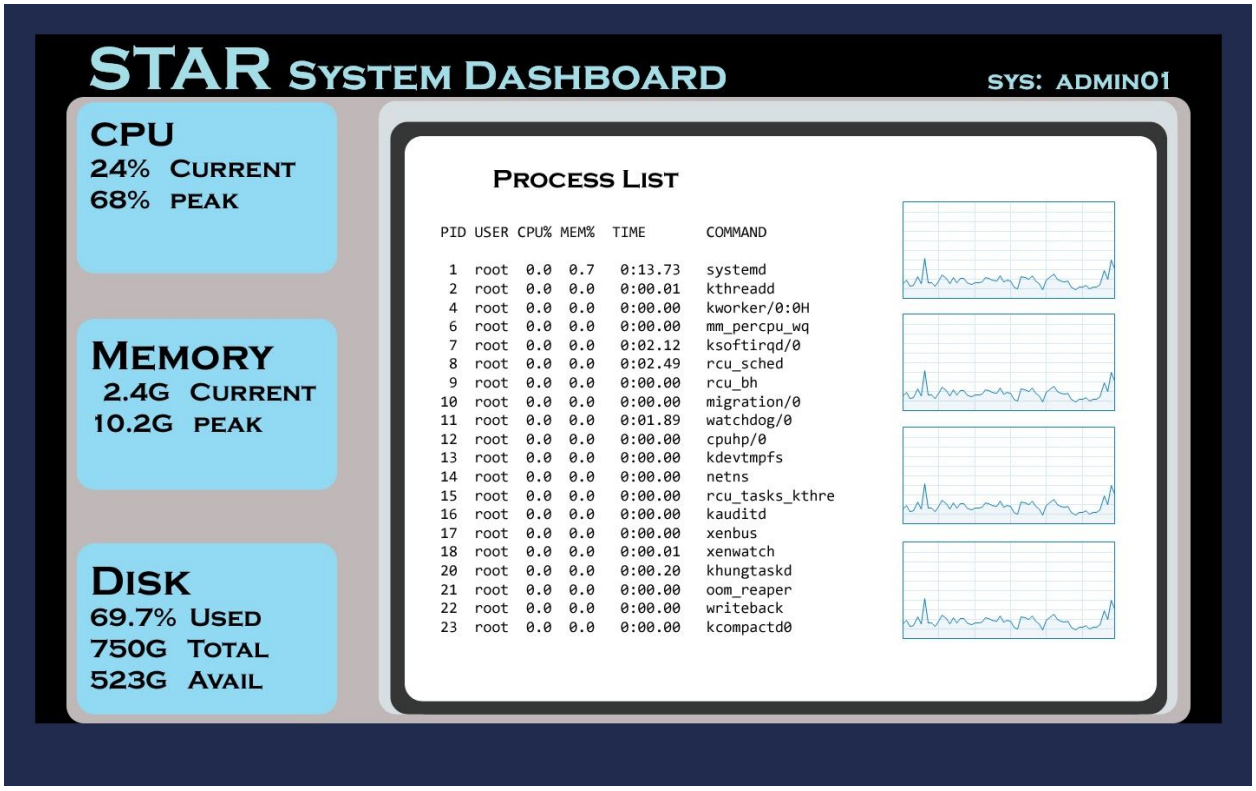
Error handling and recovery is essentially the same as if the user closes their browser; it has no effect on the server program, as data is only being read and sent to the user. Nothing is logged if the user interface stops requesting refreshed/current data.

Startup of the STAR server program and login of user or administrative users will be logged in a separate text file in a STAR program directory. This information will be available through the OS for troubleshooting, should the need arise. An email alerting capability to notify administrators of out-of-band conditions is planned for a future release. It will make use of native Linux mail capability, maintaining the use of free and open source components whenever possible.

An on-demand reporting component is planned for the initial release. The objective is to make reports available based on data already collected. Reports will be available through the program web interface.

The System Dashboard will be displayed in a web browser. The dashboard will feature certain aggregated data along the top, as summaries or averages of information collected in real time. The left side of the dashboard will display information related to CPU, memory, and disk. The right side of the display will show a list of the top twenty processes currently running on the system, based on memory utilization. A selector [drop-down menu] will be provided for user selection of display refresh interval.

STAR System Dashboard Concept



Artist concept of STAR System Dashboard

Package Build & Distribution

The Debian/Ubuntu package format, .deb, and the Advanced Package Tool, apt, are examples of a commonly used means of program distribution and installation in the Linux environment. Use of a package manager is intended to allow distribution and installation of STAR and all prerequisites via the standard Linux package installation process, using a command such as: sudo apt-get install star

As a backup, or for environments where network/Internet access is limited or non-existent, a Linux archive file in [tar, gzip] will provide an alternate delivery mechanism. This will allow distribution via optical or other removable media, if needed.

Installation

- Required authority
    - No special authority should be needed to run STAR, as commands used to generate system information do not require root privilege.
    - Elevated privilege, e.g. root via sudo command, may be needed for installation depending on system security environment.
- File permissions
    - The system will operate on the principle of least privilege. Accordingly, all Owner, Group, and Public level access will be set to the least privilege possible, consistent with required program operation.
    - The administrative role will allow updating of server program configuration.
    - The user role will allow reading of metrics data that has been collected.

Accessibility

This application will use color combinations that have necessary contrast for colorblind users to be able to use the display. Alternate text will be employed in the browser based UI to allow for use of screen reader systems. The initial release of this application will only be available in US English.

Supporting Material

Reference material will be provided via web for the following:

- Installation / configuration.
- System administration.
- General program use: User guide.
- Program updates, bug fixes, and other information.

Risks & Dependencies

Identified risk from external dependencies is low due to use of generally available open source products. Dependencies include the use of Linux distributions and their native commands, as well as specific open source application or runtime environments. Substitutes would need to be identified if a distribution were to remove a needed command or OS function, or if there were variations between distributions such as for the Cron daemon start/stop functionality.

Identified risk from internal dependencies is medium. Database access is controlled through server name and login information contained in an include file. The file is manually configured after product installation in order to reflect the customer's specific setup. Access to the file should be limited to authorized administrators, as undesirable results, resulting in a loss of program function will occur if database connection information is changed or lost. Risk mitigation exists in the form of a client backup of the database configuration file.

References

https://www.r-project.org/about.html
https://linuxacademy.com/blog/linux/the-cron-daemon/
https://sxi.io/how-to-start-stop-restart-cron-service-in-linux-centos-7
https://guides.github.com/features/issues/